Full Stack AI Application Development

Multi-Agent Research Assistant

Project Document

AI-Powered Business Intelligence Platform

Developed By

Muhammad Zain Abbas
mzabbas.bscs24seecs@seecs.edu.pk

1. Introduction and Project Overview

The Multi-Agent Research Assistant is an intelligent, full-stack application designed to automate the collection, validation, and synthesis of business intelligence data. By employing a LangGraph-based multi-agent pipeline on the backend and a reactive Next.js SPA on the frontend, the platform processes ambiguous user queries, fetches live web data, and streams formatted Markdown reports back to the user.

A critical feature of this system is its ability to recognize underspecified requests and trigger a Human-in-the-Loop (HITL) interrupt. If a query lacks necessary details (e.g., asking about a company without specifying its name), the system gracefully halts, prompts the user for clarification, and resumes its workflow seamlessly upon receiving the missing information.

Project Demonstration

Video Walkthrough: Watch the full working solution on Loom

2. System Architecture & Components

The project is decoupled into two primary domains: a robust Python backend and a dynamic React frontend.

Backend: FastAPI & LangGraph

The backend manages stateful agent workflows using Python's LangGraph and serves endpoints via FastAPI.

- **Clarity Agent:** The entry point. It evaluates if the user's query is specific enough. If the query is ambiguous, it returns a `needs_clarification` status, triggering a human interrupt.
- **Research Agent:** Activated for clear queries. It utilizes the Tavily Search API to gather deep financial and news data, outputting findings alongside a confidence score (0-10).
- **Validator Agent:** Assesses the quality and completeness of the research. If the confidence is below 6, it triggers a retry loop back to the Research Agent.
- **Synthesis Agent:** Formats the validated research into a comprehensive, user-friendly Markdown response.
- **Persistence:** An `aiosqlite` database provides checkpointing for the LangGraph state, ensuring conversational memory and reliable recovery from human interrupts.

Frontend: Next.js 16 & React 19

The user interface is a Single Page Application tailored for real-time visualization and responsiveness.

- **Server-Sent Events (SSE):** Used to stream text tokens from the backend to the UI seamlessly, ensuring a low-latency chat experience.
- **Agent Pipeline Visualizer:** A real-time graph component built with `React Flow` that highlights the currently active agent and visualizes the complex routing (including bypasses and retries).
- **State Management:** Utilizes Zustand to globally manage chat history and agent status logic without excessive re-renders.

3. Installation & Setup Instructions

To run this application locally, you will need Python 3.10+ and Node.js 18+.

Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

2. Create and activate a Python virtual environment:

```
python -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate
```

3. Install the required dependencies:

```
pip install -r requirements.txt
```

4. Create a `.env` file in the backend directory and configure the API keys:

```
GROQ_API_KEY=your_groq_key_here
TAVILY_API_KEY=your_tavily_key_here
LLM_MODEL=openai/gpt-oss-120b
CORS_ORIGINS=http://localhost:3000
SQLITE_DB_PATH=checkpoints.db
```

5. Start the FastAPI server on port 8000:

```
uvicorn app.main:app --reload --port 8000
```

Frontend Setup

1. Navigate to the frontend directory:

```
cd frontend
```

2. Install the necessary Node packages:

```
npm install
```

3. Start the Next.js development server:

```
npm run dev
```

4. Access the application in your browser at `http://localhost:3000`.

4. AI Assistance & Prompting Strategy

This project was developed by leveraging an AI coding assistant. The development process was intentionally structured to provide the AI with highly detailed, predefined architectural constraints and UI specifications.

Preparation: Documentation First

Rather than heavily prompting the AI to "write an app from scratch" and hoping for the best, the groundwork was laid by manually writing comprehensive documentation **first**. Three core specification files were created in the `docs/` folder:

1. `assignment_details.md`: Detailed the exact functional requirements, outlining the exact logic and routing rules for the 4 LangGraph agents (Clarity, Research, Validator, Synthesis).
2. `ui-spec.md`: Specified the frontend architecture, component hierarchy, state management tools (Zustand), styling framework (Vanilla CSS + CSS Variables), and the required layout structure.
3. `ui-guidelines.md`: Outlined the aesthetic requirements, enforcing a dark-mode, glassmorphic design theme with specific color palettes and micro-animations.

Execution: Simple Conversational Prompts

Because the AI agents were provided with robust, self-written documentation upfront, the actual prompting process was incredibly streamlined. Instead of complex, multi-paragraph prompt engineering, the interaction felt like a simple conversation. The AI was directed to read the provided documentation and implement it step-by-step.

Examples of the simple prompts used during development include:

- **"Read the docs folder and implement the frontend structure exactly as specified."**
- **"Now implement the backend LangGraph pipeline according to the assignment details."**
- **"The agent pipeline graph has a visual bug where the edges overlap the nodes. Fix the UI and make it collapsed by default."**

This "Docs-First" approach ensured that the AI strictly adhered to the desired architecture and aesthetic vision, reducing hallucinations and eliminating the need for excessive prompt tuning.